
**DESIGN OF A UART DMA MODULE WITH AXI BUS INTERFACE AND INTEGRATED
FIFO STRUCTURE FOR RISC-V SOC**

Huynh Vo Gia Bao, Nguyen The Bao, Ngo Thien Toan, Nguyen Thuy Hien

¹Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh, Việt Nam

* Corresponding author: Email: 23119050@hcmute.edu.vn

KEYWORDS

RISC-V;
5-stage pipeline;
AXI4-Lite; DMA;
UART;
System-on-Chip;
FPGA.

ABSTRACT

This paper presents the design and integration of a System-on-Chip (SoC) on the ZedBoard FPGA platform (Zynq XC7Z020), consisting of three hardware modules: a 5-stage pipelined RV32I RISC-V processor, a Direct Memory Access (DMA) controller, and a UART communication module with bidirectional FIFO buffers. The modules were designed using Verilog HDL and functionally verified through RTL simulation in Vivado Simulator. They were then integrated into a complete SoC through a top-level Verilog module. Simulation results demonstrate that the RISC-V processor correctly executes arithmetic, logic, memory access, and control flow instructions. The DMA controller performs AXI4 read and write transactions correctly, while the UART module successfully transmits and receives 8N1 serial frames at a baud rate of 9600 bps. The proposed SoC operates correctly with low FPGA resource utilization, providing a foundation for the development and study of RISC-V-based embedded computing systems [4] [9].

**THIẾT KẾ KHỐI UART DMA CHUẨN GIAO TIẾP BUS AXI TÍCH HỢP CẤU TRÚC FIFO
CHO HỆ THỐNG SOC RISC-V.**

Huỳnh Võ Gia Bảo, Nguyễn Thế Bảo, Ngô Thiện Toàn, Nguyễn Thúy Hiền

Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh, Việt Nam

*Tác giả liên hệ: Email: 23119050@hcmute.edu.vn

KEYWORDS

RISC-V;
pipeline 5 tầng;
AXI4-Lite; DMA;
UART;
System-on-Chip;
FPGA.

TÓM TẮT

Bài báo trình bày quá trình thiết kế và tích hợp hệ thống System-on-Chip (SoC) trên nền tảng FPGA ZedBoard (Zynq XC7Z020), bao gồm ba khối phần cứng là lõi xử lý RISC-V RV32I theo kiến trúc pipeline 5 tầng, khối điều khiển truy cập bộ nhớ trực tiếp (DMA) và khối truyền nhận nối tiếp UART tích hợp FIFO. Các khối được thiết kế bằng Verilog HDL, kiểm chứng chức năng bằng mô phỏng RTL trên Vivado Simulator, sau đó được tích hợp trong module Verilog để tạo thành hệ thống SoC hoàn chỉnh. Kết quả mô phỏng cho thấy lõi RISC-V thực hiện đúng các lệnh số học, logic, truy cập bộ nhớ và điều khiển luồng, khối DMA thực hiện đúng các giao dịch đọc và ghi theo chuẩn AXI4, trong khi khối UART truyền và nhận dữ liệu theo chuẩn 8N1 ở tốc độ 9600 bps. Hệ thống hoạt động đúng chức năng và sử dụng tài nguyên FPGA ở mức thấp, tạo nền tảng cho việc mở rộng và nghiên cứu các hệ thống SoC RISC-V trong lĩnh vực máy tính nhúng [4] [9].

1. GIỚI THIỆU

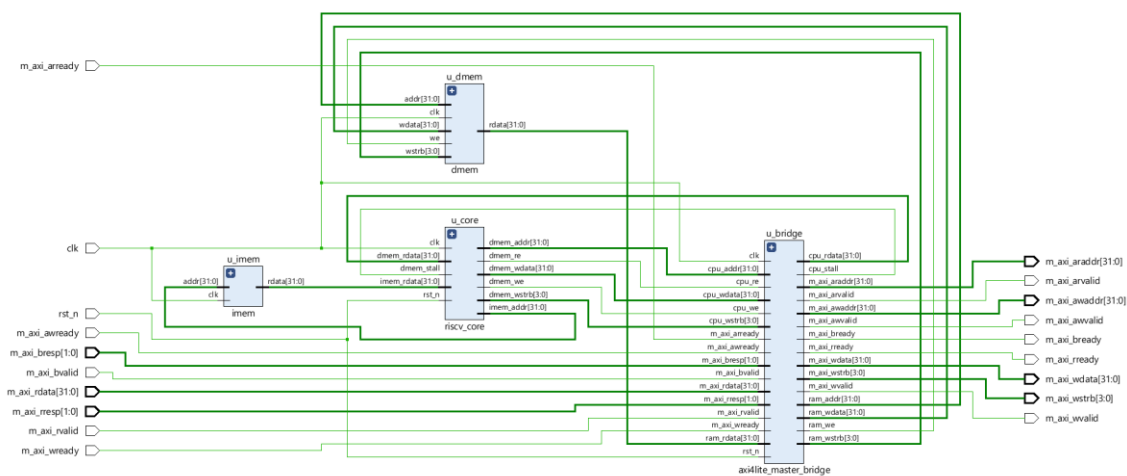
Kiến trúc tập lệnh mở RISC-V ngày càng được dùng rộng rãi trong nghiên cứu và đào tạo thiết kế vi xử lý nhờ tính mở và khả năng mở rộng linh hoạt. Tự thiết kế lõi RV32I pipeline nhiều tầng là bước đi cơ bản để làm chủ nguyên lý CPU hiện đại, đồng thời là nền tảng xây dựng SoC tùy biến trên FPGA [1][2]. Một SoC thực tế cần thêm các khối ngoại vi hỗ trợ trao đổi dữ liệu tốc độ cao và giao tiếp bên ngoài. Vì vậy, nghiên cứu này thiết kế đồng thời ba khối trên FPGA ZedBoard (Zynq XC7Z010): lõi RV32I pipeline 5 tầng giao tiếp qua AXI4-Lite Master, khối DMA đóng vai trò AXI4 Master, truyền dữ liệu giữa bộ nhớ và ngoại vi mà không cần CPU can thiệp từng byte, và khối UART có FIFO đệm, giao tiếp AXI4-Stream, dùng xuất dữ liệu tốc độ thấp. Đóng góp chính của bài báo là tích hợp ba khối thành một hệ thống thống nhất, làm rõ luồng dữ liệu và cơ chế phối hợp giữa CPU, DMA, UART qua bus AXI chung - khác với các nghiên cứu trước chỉ dùng ở thiết kế đơn khối.

2. PHƯƠNG PHÁP VÀ THIẾT KẾ HỆ THỐNG

Phương pháp nghiên cứu thực hiện theo hướng thiết kế phần cứng số bằng Verilog, kết hợp mô phỏng chức năng trên Vivado Simulator để xác minh tính đúng đắn của thiết kế trước khi tổng hợp lên FPGA. Quy trình gồm: (1) Xây dựng kiến trúc tổng thể và xác định chức năng từng khối; (2) Hiện thực từng khối chức năng độc lập; (3) Xây dựng testbench cho từng khối; và (4) Tích hợp ba khối qua AXI Interconnect và kiểm thử toàn hệ thống.[5]

2.1. Thiết kế lõi xử lý RISC-V

Lõi xử lý RISC-V được xây dựng theo kiến trúc pipeline 5 tầng, thực thi tập lệnh chuẩn RV32I. Toàn bộ được đóng gói trong 1 module gồm 4 thành phần chính: lõi CPU, bộ nhớ lệnh, bộ nhớ dữ liệu và cầu nối giao tiếp AXI4-Lite.



Hình 1: Sơ đồ RTL kiến trúc khối SoC RISC-V

Hình trên là sơ đồ RTL RISC-V, gồm bốn khối chính: u_imem là ROM lệnh, cung cấp lệnh 32bit cho CPU mỗi chu kỳ clock, u_core là lõi CPU pipeline 5 tầng, xử lý lệnh và phát sinh yêu cầu đọc/ghi dữ liệu ra hai hướng tùy theo địa chỉ, u_dmem (RAM nội bộ), u_bridge (định tuyến) được dùng để giao tiếp với ngoại vi bên ngoài (UART, DMA) qua chuẩn AXI4-Lite. Khi đang chờ phản hồi từ ngoại vi, u_bridge tự động dừng pipeline để CPU không bỏ sót lệnh nào.

Mỗi lệnh đi qua 5 công đoạn nối tiếp nhau: IF đọc lệnh từ bộ nhớ, ID giải mã lệnh và đọc thanh ghi, EX thực thi phép tính bằng ALU, MEM đọc/ghi bộ nhớ dữ liệu, WB ghi kết quả về thanh ghi đích. Nhờ pipeline, tại mỗi chu kỳ clock có thể có tới 5 lệnh đang thực thi đồng thời ở các công đoạn khác nhau, tăng đáng kể tốc độ xử lý so với kiến trúc đơn chu kỳ.

Bảng 1: Tập lệnh RV32I được hỗ trợ bởi bộ xử lý

Nhóm lệnh	Các lệnh được hỗ trợ
R-type	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
I-type (tính toán)	ADDI, XORI, ORI, ANDI, SLTI, SLTIU, SLLI, SRLI, SRAI
Load / Store	LB, LH, LW, LBU, LHU / SB, SH, SW
Branch / Jump	BEQ, BNE, BLT, BGE, BLTU, BGEU / JAL, JALR
Upper Immediate	LUI, AUIPC

Pipeline gặp 3 loại xung đột và tất cả được xử lý hoàn toàn tự động bằng phần cứng. Load-use hazard xảy ra khi lệnh LOAD theo sau ngay bởi lệnh cần dùng kết quả, CPU tự dừng 1 chu kỳ để chờ dữ liệu về. Branch/Jump hazard xảy ra khi lệnh rẽ nhánh hoặc nhảy được xác định tại tầng EX, CPU tự xóa 2 lệnh đã đọc nhằm trước đó tránh thực thi sai. AXI stall xảy ra khi đang chờ phản hồi từ ngoại vi, toàn bộ pipeline đứng yên cho đến khi nhận được kết quả.

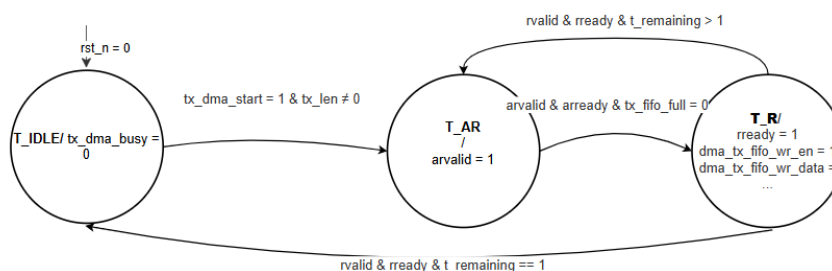
Để giảm thiểu số chu kỳ dừng, forwarding unit tự động lấy kết quả trung gian từ tầng EX hoặc MEM chuyển thẳng về đầu vào ALU của lệnh kế tiếp mà không cần chờ ghi về thanh ghi. Nhờ đó hầu hết các trường hợp xung đột dữ liệu được giải quyết ngay mà không làm chậm pipeline.

Khi CPU cần truy cập ngoại vi, cầu nối axi4lite tự động phân biệt hai vùng địa chỉ: dưới 16KB truy cập RAM nội bộ ngay lập tức trong 1 chu kỳ, trên 16KB chuyển sang giao tiếp với ngoại vi qua bus AXI4-Lite. Trong thời gian chờ ngoại vi phản hồi, toàn bộ pipeline được giữ nguyên để không bỏ sót lệnh nào.

2.2. Thiết kế khối DMA

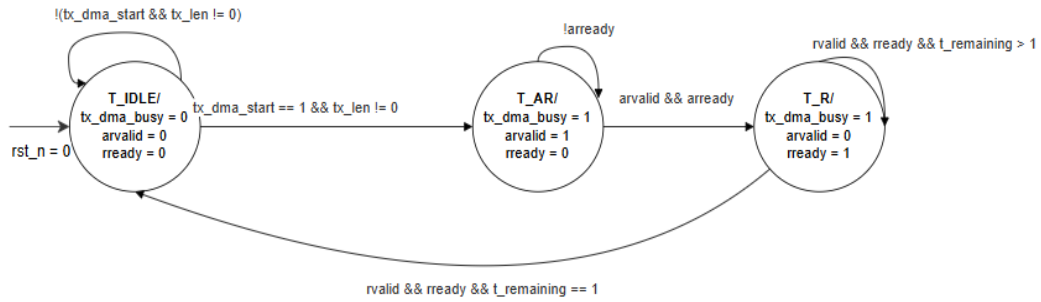
Khối DMA truyền dữ liệu tự động giữa bộ nhớ hệ thống và khối UART mà không cần CPU can thiệp từng byte. Sau khi CPU cấu hình địa chỉ, độ dài và bật lệnh (qua thanh ghi CTRL của UART), DMA tự xử lý hoàn toàn phần còn lại. Khối có 2 FSM hoàn toàn độc lập :

FSM TX tự động chuyển từng byte từ bộ nhớ vào hàng đợi UART để phát đi, không cần CPU can thiệp từng byte. Khi chờ lệnh, DMA đứng yên. Khi CPU ra lệnh, DMA gửi yêu cầu đọc lên bus khi hàng chờ còn chỗ, nếu đầy thì chờ. Sau khi đọc được dữ liệu, DMA chọn đúng byte cần thiết và đẩy vào hàng chờ. Quá trình lặp lại cho đến khi hết số byte cần gửi rồi báo hoàn thành.



Hình 2: Sơ đồ FSM TX

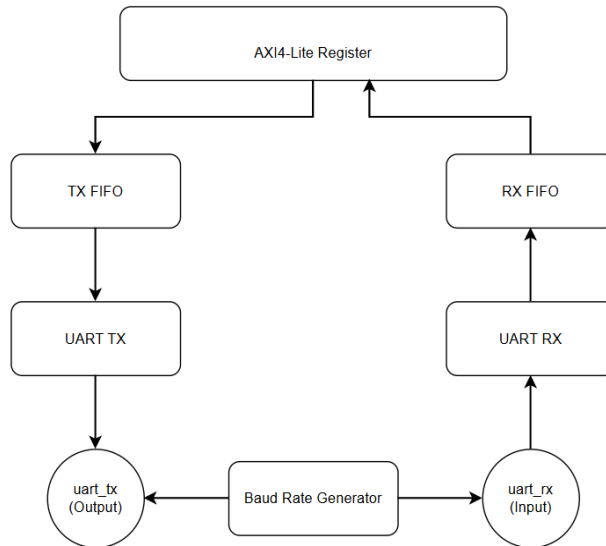
FSM RX tự động lấy từng byte UART vừa thu được và ghi vào bộ nhớ. Cần thêm một bước so với TX vì ghi vào bộ nhớ phức tạp hơn đọc, phải chờ xác nhận ghi xong mới tiếp tục. Khi chờ lệnh, DMA đứng yên. Khi CPU ra lệnh, DMA chờ hàng đợi có dữ liệu rồi gửi địa chỉ ghi lên bus, tiếp theo gửi dữ liệu, rồi chờ xác nhận ghi thành công. Sau khi được xác nhận, nếu còn byte thì tiếp tục, nếu hết thì báo hoàn thành.



Hình 3: Sơ đồ FSM RX

2.3. Thiết kế khối UART

Khối UART gồm 6 module con hoạt động phối hợp với nhau: bộ tạo tần số baud, bộ phát TX, bộ thu RX, hai FIFO và bộ thanh ghi điều khiển AXI-Lite Register.



Hình 4: Kiến trúc tổng thể khối UART

Bộ tạo tần số baud: Bộ đếm 16-bit đếm từ 0 đến baud_div. Khi đạt giá trị baud_div, bộ đếm reset về 0 và phát xung tick_x16=1 trong đúng 1 chu kỳ clock.

Công thức tính baud_div:

$$\text{baud_div} = f_{\text{CLK}} / (16 \times \text{BAUD_RATE}) - 1[7].$$

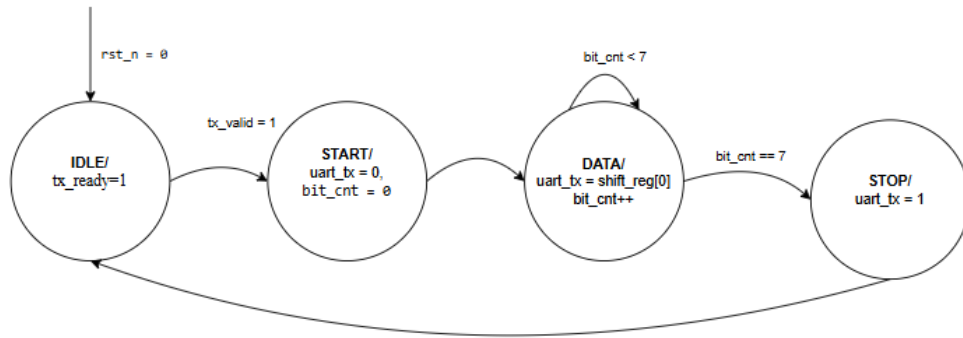
Với tần số 100MHz: $100\text{MHz} / (16 \times 9600) - 1 = 651$

AXI-Lite Register: Là cầu nối giữa CPU và toàn bộ khối UART, chứa 11 thanh ghi (32 bit). Gồm 2 FSM ghi và đọc với cơ chế handshake để nhận lệnh từ CPU, nhờ đó CPU đọc/ghi vào các thanh ghi để điều khiển toàn bộ UART.

Bảng 2: Địa chỉ các thanh ghi

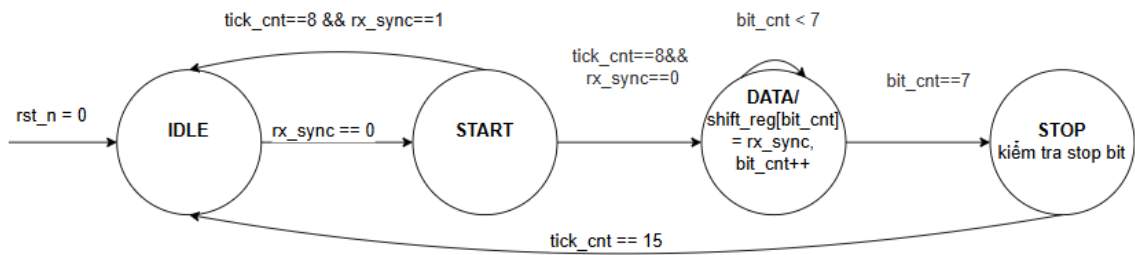
Offset	Tên	Kiểu	Mô tả chi tiết
0x00	CTRL	R/W	bit0 TX_START . bit1 RX_START . bit2 SOFT_RST . bit3 TX_FIFO_CLR . bit4 RX_FIFO_CLR tự xóa về 0 sau khi DMA nhận lệnh
0x04	STATUS	RO	bit0 TX_BUSY . bit1 RX_BUSY . bit2 TXFIFO_EMPTY . bit3 TXFIFO_FULL . bit4 RXFIFO_EMPTY . bit5 RXFIFO_FULL
0x08	BAUD_DIV	R/W	Hệ số chia baud = $f_CLK/(16 \times BAUD) - 1$. Mặc định: 651
0x0C	TX_ADDR	R/W	Địa chỉ nguồn trong bộ nhớ cho DMA TX
0x10	TX_LEN	R/W	Số byte cần phát qua DMA TX
0x14	RX_ADDR	R/W	Địa chỉ đích trong bộ nhớ cho DMA RX
0x18	RX_LEN	R/W	Số byte cần nhận qua DMA RX
0x1C	IRQ_EN	R/W	bit0 TX_DONE . bit1 RX_DONE . bit2 RX_OVF . bit3 RX_FERR
0x20	IRQ_STATUS	R/W	Sticky flag, ghi 1 để xóa từng bit
0x24	TX_DATA	WO	Ghi byte trực tiếp vào TX FIFO (polling, không qua DMA)
0x28	RX_DATA	RO	Đọc lấy 1 byte từ RX FIFO

Bộ phát TX: Có nhiệm vụ lấy từng byte từ TX FIFO và chuyển thành tín hiệu chuẩn UART 8N1, mỗi byte được đóng gói thành 10 bit gồm 1 bit start, 8 bit dữ liệu (LSB trước) và 1 bit stop. Bộ phát hoạt động theo FSM 4 trạng thái, mỗi trạng thái kéo dài 1 chu kỳ baud. Ở trạng thái IDLE, đường TX giữ mức cao và chờ TX FIFO có dữ liệu. Khi có byte, chuyển sang trạng thái START để kéo đường TX xuống thấp báo hiệu bắt đầu khung. Tiếp theo ở trạng thái DATA, 8 bit dữ liệu được phát lần lượt LSB trước, mỗi bit giữ đúng 1 chu kỳ baud. Cuối cùng ở trạng thái STOP, đường TX kéo lên cao trong 1 chu kỳ baud để kết thúc khung rồi quay về trạng thái IDLE chờ byte tiếp theo.



Hình 5: Sơ đồ FSM UART TX

Bộ thu UART: Nhận tín hiệu nối tiếp từ chân RX và ghép lại thành từng byte để đưa vào hàng đợi RX. Khi chân RX bị kéo xuống mức thấp, bộ thu biết có dữ liệu đến và bắt đầu đếm thời gian. Thay vì đọc giá trị ngay lập tức, bộ thu đợi đến đúng giữa chu kỳ bit mới đọc, vì lúc đó tín hiệu ổn định nhất, tránh đọc nhầm khi tín hiệu đang chuyển mức ở hai đầu bit. Cứ mỗi chu kỳ bit lại đọc một lần, lặp lại 8 lần để thu đủ 1 byte. Sau khi thu đủ 8 bit, bộ thu kiểm tra stop bit cuối cùng, nếu chân RX lên cao đúng như kỳ vọng thì byte hợp lệ và được đẩy vào hàng đợi, nếu không thì báo lỗi và bỏ qua byte đó.

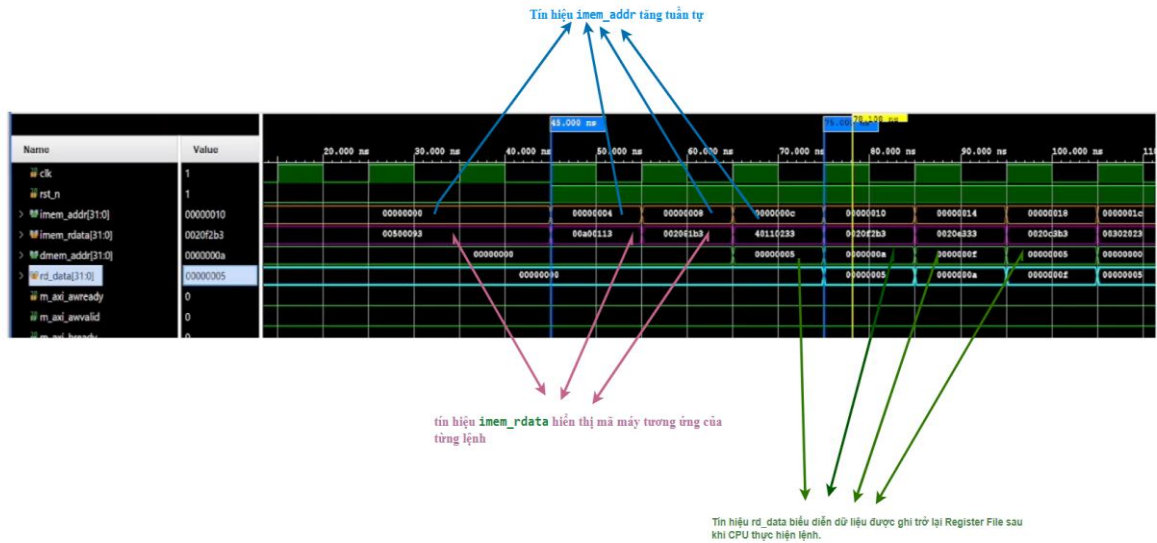


Hình 6: Sơ đồ FSM UART RX

FIFO đồng bộ (DATA_WIDTH=8, DEPTH=16). FIFO sử dụng một mảng nhớ mem[0:DEPTH-1] để lưu dữ liệu cùng hai con trỏ là wr_ptr (con trỏ ghi) và rd_ptr (con trỏ đọc). Hai con trỏ này có độ rộng ADDR_WIDTH + 1 bit, trong đó các bit thấp dùng để xác định địa chỉ ô nhớ, còn bit MSB được thêm vào để theo dõi việc con trỏ đã quay vòng qua bộ nhớ hay chưa. Cụ thể, FIFO rỗng khi wr_ptr và rd_ptr hoàn toàn bằng nhau (bao gồm cả bit MSB), còn FIFO đầy khi hai con trỏ có cùng địa chỉ nhưng bit MSB khác nhau, cho thấy con trỏ ghi đã vượt trước con trỏ đọc đúng một vòng. Số lượng dữ liệu hiện có trong FIFO được xác định bằng phép trừ không dấu wr_ptr - rd_ptr.

3. KẾT QUẢ VÀ THẢO LUẬN

3.1 Kết quả mô phỏng lõi RISC-V



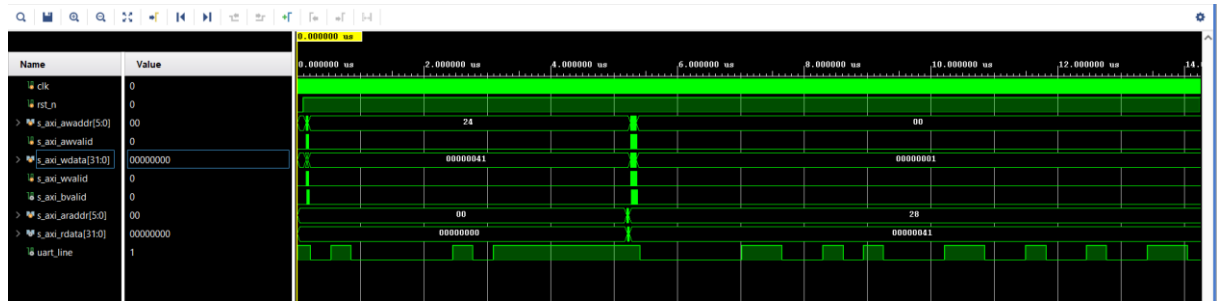
Hình 7: Kết quả mô phỏng lõi RISC-V

Trong khoảng thời gian từ 20 ns đến khoảng 100 ns, CPU bắt đầu thực thi các lệnh đầu tiên được nạp từ bộ nhớ chương trình. Tín hiệu `imem_addr` tăng tuần tự theo từng chu kỳ xung clock (0x00, 0x04, 0x08, 0x0C, ...), cho thấy CPU đang lần lượt lấy các lệnh từ Instruction Memory. Đồng thời, tín hiệu `imem_rdata` hiển thị mã máy tương ứng của từng lệnh. Tín hiệu `rd_data` biểu diễn dữ liệu được ghi trở lại Register File sau khi CPU thực hiện lệnh. Trong giai đoạn kiểm thử các lệnh số học và logic, giá trị của `rd_data` cũng chính là kết quả tính toán của ALU.

Bảng 3: Kết quả kiểm thử theo từng nhóm lệnh

Địa chỉ	Mã lệnh	Chức năng	Kết quả
0x00	500093	<code>addi x1, x0, 5</code>	<code>x1 = 0x00000005</code>
0x04	00A00113	<code>addi x2, x0, 10</code>	<code>x2 = 0x0000000A</code>
0x08	002081B3	<code>add x3, x1, x2</code>	<code>x3 = 0x0000000F</code>
0x0C	40110233	<code>sub x4, x2, x1</code>	<code>x4 = 0x00000005</code>
0x10	0020F2B3	<code>and x5, x1, x2</code>	<code>x5 = 0x00000000</code>
0x14	0020E333	<code>or x6, x1, x2</code>	<code>x6 = 0x0000000F</code>
0x18	0020C3B3	<code>xor x7, x1, x2</code>	<code>x7 = 0x0000000F</code>

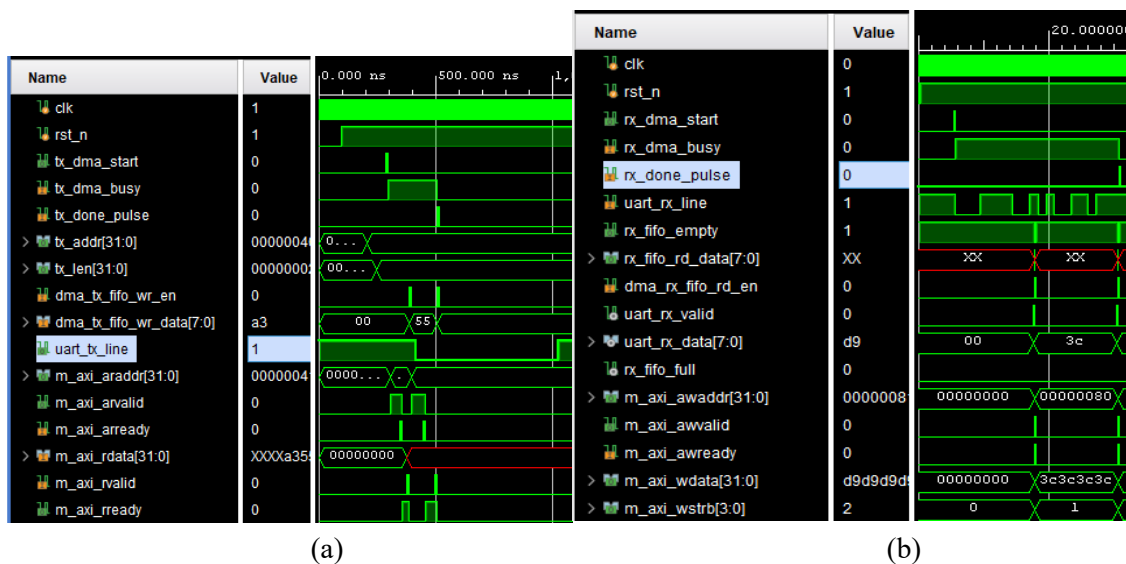
3.2 Kết quả mô phỏng UART



Hình 8: Dạng sóng truyền dữ liệu UART khi phát ký tự 'A'.

CPU ghi 1 byte qua TX_DATA rồi đọc lại qua RX_DATA theo chế độ loopback (chân TX nối vào chân RX). Đầu tiên CPU thực hiện ghi với địa chỉ s_axi_awaddr = 0x24 và s_axi_wdata = 0x00000041 (ký tự 'A'). Ngay sau đó uart_line phát từng bit của byte 0x41 ra đường truyền. Vì TX và RX được nối loopback, UART đồng thời thu lại chính byte vừa phát và đẩy vào RX FIFO. Sau khoảng 500 chu kỳ clock, CPU đọc lại với s_axi_araddr = 0x28 (địa chỉ thanh ghi RX_DATA) và nhận về s_axi_rdata = 0x00000041.

3.3 Kết quả mô phỏng DMA



Hình : Dạng sóng Ghi (a) và Đọc (b) của DMA

4. TÍCH HỢP HỆ THỐNG

4.1 Hai hướng tiếp cận

Nhóm đã thử nghiệm 2 hướng tích hợp toàn hệ thống:

Hướng 1: Vivado IP Integrator GUI (THẤT BẠI)

Đóng gói từng khối thành các IP riêng, sau đó kết nối chúng thông qua giao tiếp AXI để hình thành hệ thống SoC hoàn chỉnh nhưng không thành công do Vivado tự chèn thêm các khối AXI Protocol Converter và crossbar giữa CPU và UART. Các khối tự sinh này không forward giao dịch AXI đúng cách, CPU bị treo vĩnh viễn, UART không nhận được lệnh nào. Nhóm đã thử nhiều hướng sửa trong Block Design: điều chỉnh độ rộng địa chỉ AXI, bổ sung tín hiệu burst, thêm AXI Protocol Converter thủ công, kết nối điểm-điểm bỏ qua Interconnect, tất cả đều gặp vấn đề về type mismatch.

Hướng 2 : Viết file Verilog kết nối trực tiếp (THÀNH CÔNG)

Thay vì dùng Block Design GUI, nhóm viết 1 file Verilog top-level (soc_top.v) kết nối tay toàn bộ các module bằng wire, không qua bất kỳ tầng trung gian nào của Vivado:

CPU <-> UART: cổng m_axi của CPU (riscv_soc_top) nối thẳng vào cổng s_axi của UART (uart_axi_top) bằng 17 wire trực tiếp (awaddr, awvalid, awready, wdata, wstrb, wvalid, wready, bresp, bvalid, bready, araddr, arvalid, arready, rdata, rresp, rvalid, rready), theo chuẩn AXI4-Lite.

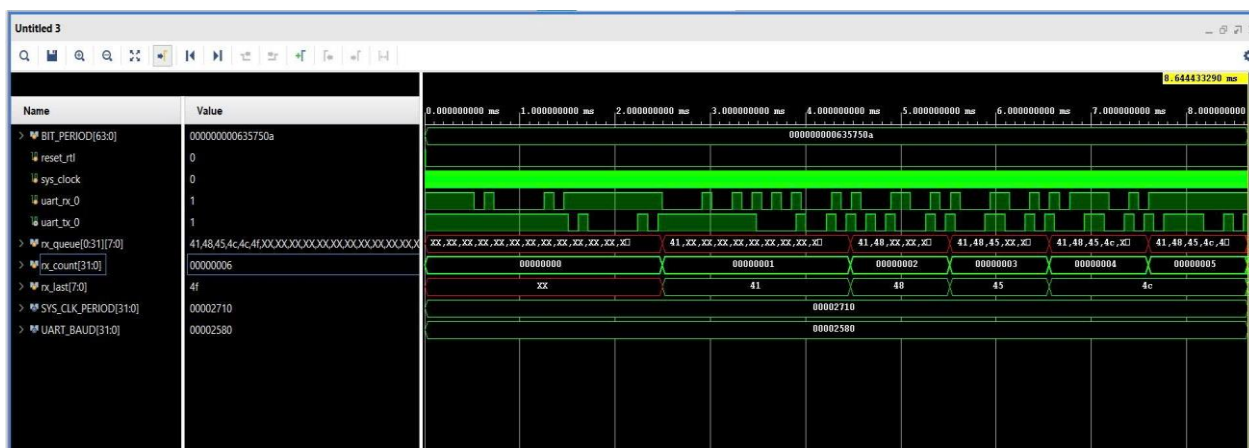
UART <-> DMA: toàn bộ 14 tín hiệu điều phối (tx_dma_start, tx_addr, tx_len, tx_fifo_full, rx_dma_start, rx_addr, rx_len, rx_fifo_rd_data, rx_fifo_empty, tx_start_clear, tx_dma_busy, tx_done_pulse, dma_tx_fifo_wr_en/data, rx_start_clear, rx_dma_busy, rx_done_pulse, dma_rx_fifo_rd_en) nối trực tiếp bằng wire

DMA m_axi (AXI4 master): kết nối trực tiếp tới một khối Dummy BRAM nội bộ (sys_ram, RAM 4KB tự viết trong soc_top.v) đóng vai trò bộ nhớ hệ thống cho DMA đọc/ghi.

4.2 Kết quả mô phỏng tích hợp

Bảng 4: Các trường hợp kiểm tra

Trường hợp kiểm tra	Byte gửi	Byte nhận
Echo 1 byte	0x41 ('A')	0x41 ('A')
Echo chuỗi 'H'	0x48	0x48
Echo chuỗi 'E'	0x45	0x45
Echo chuỗi 'L'	0x4C	0x4C
Echo chuỗi 'L'	0x4C	0x4C
Echo chuỗi 'O'	0x4F	0x4F



Hình 16: Kết quả sau khi chạy mô phỏng

Testbench gửi lần lượt các byte vào chân uart_rx_0 và theo dõi phản hồi echo trên uart_tx_0. Tín hiệu rx_count tăng dần từ 0 lên 5 qua từng mốc thời gian, mỗi lần tăng tương ứng với 1 byte được monitor thu về từ uart_tx_0. Tín hiệu rx_last lần lượt hiển thị 0x41 -> 0x48 -> 0x45 -> 0x4C -> 0x4F, đúng thứ tự các byte đã gửi vào. Tín hiệu rx_queue tích lũy dần các byte nhận được theo đúng thứ tự.

4.3 Phân tích, đánh giá tài nguyên sử dụng của hệ thống

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	Bonded IOB (200)	BUFGCTRL (32)
▼ N soc_top	4231	2379	1174	513	1517	2183	2048	1	4	1
> I u_cpu (riscv_soc_top)	3725	1619	1174	513	1302	1677	2048	0	0	0
I u_dma (axi4_dma_master)	194	147	0	0	81	194	0	0	0	0
> I u_uart (uart_axi_top)	312	612	0	0	200	312	0	0	0	0

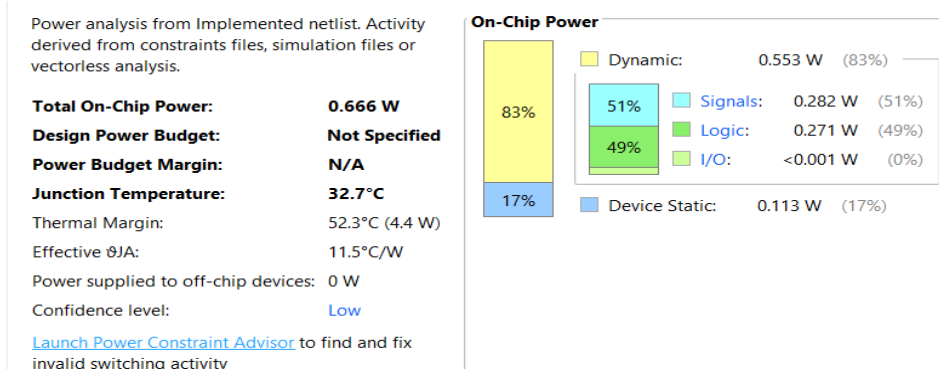
Hình 17: tài nguyên sử dụng của hệ thống

Khối RISC-V : Chiếm phần lớn không gian logic với 3725 LUTs. Ngoài ra, bộ nhớ lệnh và bộ nhớ dữ liệu cục bộ hiện đang được tổng hợp dưới dạng Distributed RAM (thể hiện qua con số 2048 LUTs as Memory).

Khối UART: Khối ngoại vi sử dụng 312 LUTs và 610 thanh ghi.

Khối DMA: Dùng 194 LUTs và 147 thanh ghi..

Tổng LUT và Register sử dụng chiếm chưa tới 10% so với tổng tài nguyên của dòng chip FPGA cho thấy kiến trúc được thiết kế gọn nhẹ, đáp ứng đúng yêu cầu của một SoC nhúng cơ bản, đồng thời để lại phần dự trữ lớn cho các bản nâng cấp.



Hình 18: Công suất sử dụng của khối tổng

Tổng công suất trên chip (0.666 W): Đây là mức tiêu thụ năng lượng cực kỳ thấp và hoàn toàn hợp lý đối với một kiến trúc SoC lõi đơn (RISC-V + UART + DMA) chiếm khoảng 4200 LUTs. Nó cho thấy thiết kế rất tiết kiệm điện năng.

Nhiệt độ hoạt động (32.7°C): Mức nhiệt này chỉ nhỉnh hơn nhiệt độ phòng một chút. Chip hoạt động rất mát mẻ, an toàn tuyệt đối.

Phân bố công suất

Công suất động (Dynamic Power - 0.553 W, chiếm 83%): Đây là năng lượng tiêu hao khi mạch thực sự làm việc (chuyển đổi trạng thái 0-1). Trong đó, nó chia đều cho việc truyền dẫn tín hiệu trên dây (Signals - 51%) và hoạt động tính toán của các cổng logic (Logic - 49%). Điều này phản ánh đúng đặc thù của một vi xử lý RISC-V đang liên tục chạy lệnh và luân chuyển dữ liệu qua bus AXI4.

Công suất tĩnh (Device Static - 0.113 W, chiếm 17%): Đây là công suất rò rỉ tự nhiên của bản thân con chip FPGA khi được cấp điện, dù mạch có đang chạy hay không. Con số 0.113 W là mức rò rỉ tiêu chuẩn của dòng chip Zynq-7000.

BRAM và I/O (< 0.001 W): Vì hệ thống chỉ dùng 1 khối BRAM giả lập cỡ nhỏ và chỉ có 2 chân I/O hoạt động (TX, RX), mức tiêu thụ năng lượng ở đây nhỏ đến mức gần như bằng không.

5. KẾT LUẬN

Bài báo đã trình bày quá trình thiết kế, kiểm thử và tích hợp ba khối phần cứng: lõi xử lý RISC-V RV32I pipeline 5 tầng, khối DMA và khối UART thành một hệ thống hoàn chỉnh. Kết quả mô phỏng RTL cho thấy từng khối hoạt động đúng thiết kế: lõi RISC-V đạt 18/18 phép kiểm tra, DMA hoàn tất chu trình đọc/ghi AXI4, UART truyền đúng khung 8N1 với sai số baud nhỏ.. Hướng phát triển tiếp theo là kiểm thử tích hợp trên giao dịch AXI thực tế với bộ nhớ ngoài, và bổ sung thêm các ngoại vi khác vào cùng kiến trúc SoC.

TÀI LIỆU THAM KHẢO

- [1] A. Waterman and K. Asanović, Eds., The RISC-V Instruction Set Manual, Volume I: User-Level ISA, RISC-V Foundation, 2019.
- [2] D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware/Software Interface, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2020.
- [3] ARM Limited, "AMBA AXI and ACE Protocol Specification," ARM IHI 0022, 2021.
- [4] Xilinx Inc., "Zynq-7000 SoC Technical Reference Manual," UG585, 2021.
- [5] Xilinx Inc., "Vivado Design Suite User Guide: Synthesis," UG901, 2022.
- [6] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2017.
- [7] Telecommunications Industry Association, "Interface Between Data Terminal Equipment and Data Circuit-Terminating Equipment Employing Serial Binary Data Interchange," TIA/EIA-232-F, 1997.
- [8] ARM Ltd., "AMBA AXI4-Stream Protocol Specification," IHI0051A, ARM Holdings, 2010.
- [9] S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.
- [10] Xilinx Inc., "AXI UART 16550 v2.0 Product Guide," PG143, 2017.

Nguyen The Bao Currently pursuing Bachelor degree in Computer Engineering at Ho Chi Minh City University of Technology and Education, Vietnam.

Research interests: embedded systems, IoT, robotics, design systems, semiconductors.

Huynh Vo Gia Bao Currently pursuing Bachelor degree in Computer Engineering at Ho Chi Minh City University of Technology and Education, Vietnam

Research interests: embedded systems, IoT, AI.

Nguyen Thuy Hien Currently pursuing Bachelor degree in Computer Engineering at Ho Chi Minh City University of Technology and Education, Vietnam

Research interests: embedded systems, semiconductor.

Ngo Thien Toan Currently pursuing Bachelor degree in Computer Engineering at Ho Chi Minh City University of Technology and Education, Vietnam

Research interests: embedded systems, IoT, AI.

LINK GITHUB: [GitHub - Hardware-Software-Codesign-ST2/FINAL-PROJECT · GitHub](#)

LINK VIDEO: <https://www.youtube.com/watch?v=otC6dJ1Bdi8>



HCM-UTE

TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

HCMC University of Technology and Engineering

THIẾT KẾ KHỎI UART DMA CHUẨN GIAO TIẾP BUS AXI TÍCH HỢP CẤU TRÚC FIFO CHO HỆ THỐNG SOC RISC-V.



Nội dung

1. Giới thiệu bài toán và mục tiêu thiết kế khối UART DMA tích hợp trong hệ thống SoC RISC-V.
2. Thiết kế kiến trúc phần cứng gồm các khối: UART TX/RX, FIFO, DMA TX/RX, AXI4-Lite Register và AXI4 Master.
3. Xây dựng và mô phỏng hoạt động của từng khối bằng Verilog HDL trên Vivado.
4. Đóng gói các module thành IP và tích hợp vào hệ thống SoC RISC-V
5. Đánh giá kết quả thông qua mô phỏng chức năng, mức sử dụng tài nguyên FPGA và khả năng mở rộng của hệ thống.

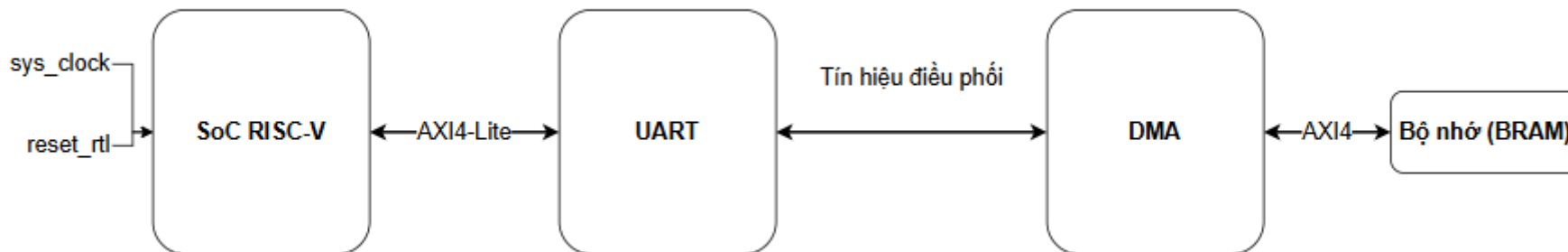
Giới thiệu - Đề xuất

Giới thiệu đề tài

Trong các hệ thống nhúng hiện đại, UART là giao thức truyền thông nối tiếp phổ biến được dùng để debug, ghi log và trao đổi dữ liệu giữa các thiết bị. Tuy nhiên khi cần truyền một lượng lớn dữ liệu, nếu để CPU tự xử lý từng byte sẽ tiêu tốn nhiều tài nguyên xử lý và làm giảm hiệu năng toàn hệ thống. Đề tài này hướng đến việc thiết kế một hệ thống SoC hoàn chỉnh tích hợp lõi xử lý RISC-V, khối UART và khối DMA trên cùng một chip FPGA

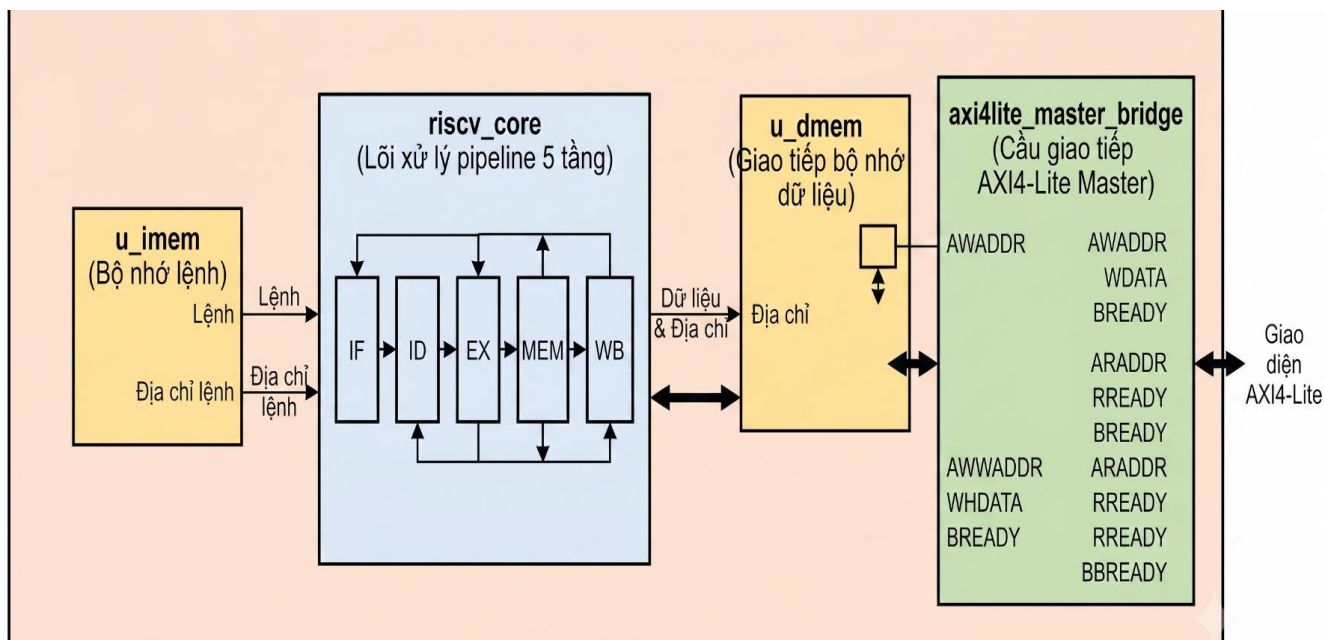
Đề xuất thiết kế

Hệ thống được đề xuất gồm 3 khối chính kết nối với nhau như sơ đồ. Khối SoC RISC-V đóng vai trò trung tâm điều khiển, thực thi firmware và giao tiếp với ngoại vi qua chuẩn AXI4-Lite. Khối UART thực hiện việc phát và thu dữ liệu trên đường truyền vật lý, đồng thời cung cấp giao diện thanh ghi để CPU cấu hình và điều khiển. Khối DMA kết nối trực tiếp với UART qua tín hiệu điều phối nội bộ và với bộ nhớ hệ thống qua chuẩn AXI4, cho phép truyền dữ liệu tự động mà không cần CPU can thiệp.





Sơ đồ khối – SOC RISC-V



Sơ đồ thể hiện kiến trúc tổng thể của khối SoC RISC-V gồm 4 module con kết nối với nhau.

ROM lệnh lưu chương trình firmware được nạp sẵn từ file hex. CPU đọc lệnh từ đây mỗi chu kỳ clock theo địa chỉ PC hiện tại.

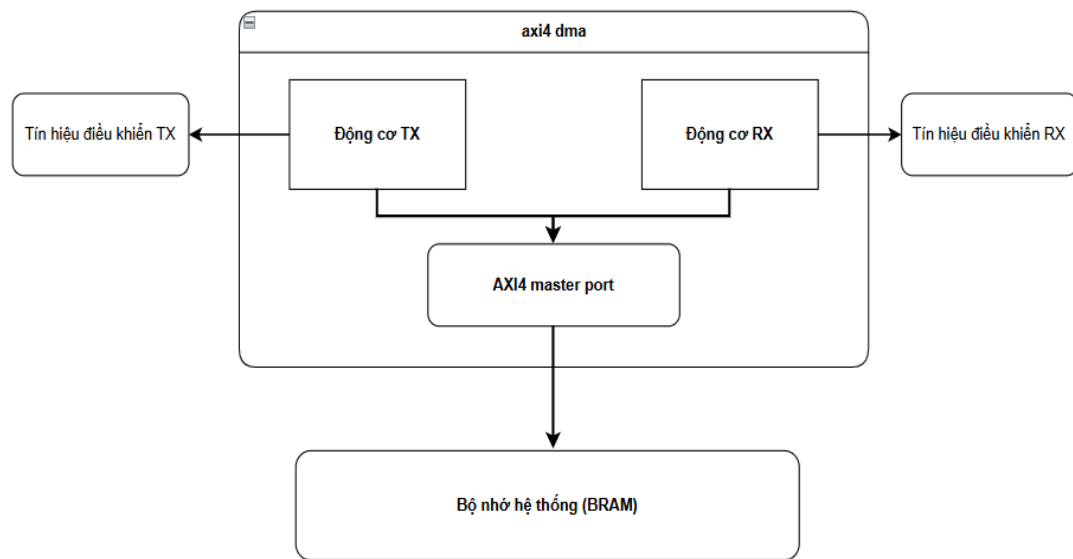
CPU pipeline 5 tầng là lõi xử lý trung tâm, thực thi tập lệnh RV32I qua 5 tầng IF -> ID -> EX -> MEM -> WB. Đây là khối kết nối với tất cả các thành phần còn lại.

RAM dữ liệu lưu trữ dữ liệu tạm thời trong quá trình chương trình chạy. CPU đọc và ghi trực tiếp vào đây khi địa chỉ nằm trong vùng 16KB nội bộ.

Cầu nối AXI4-Lite phân biệt địa chỉ nội bộ và địa chỉ ngoại vi. Khi CPU truy cập địa chỉ vượt quá 16KB, cầu nối tự động chuyển yêu cầu thành giao dịch AXI4-Lite và gửi ra ngoài để giao tiếp với khối UART.



Sơ đồ khối – DMA



- Có vai trò giảm tải cho CPU
- Nếu không có DMA:
 - Ghi từng byte một vào thanh ghi TX_DATA và chờ UART phát xong rồi mới ghi byte tiếp theo
 - CPU không làm được việc gì khác

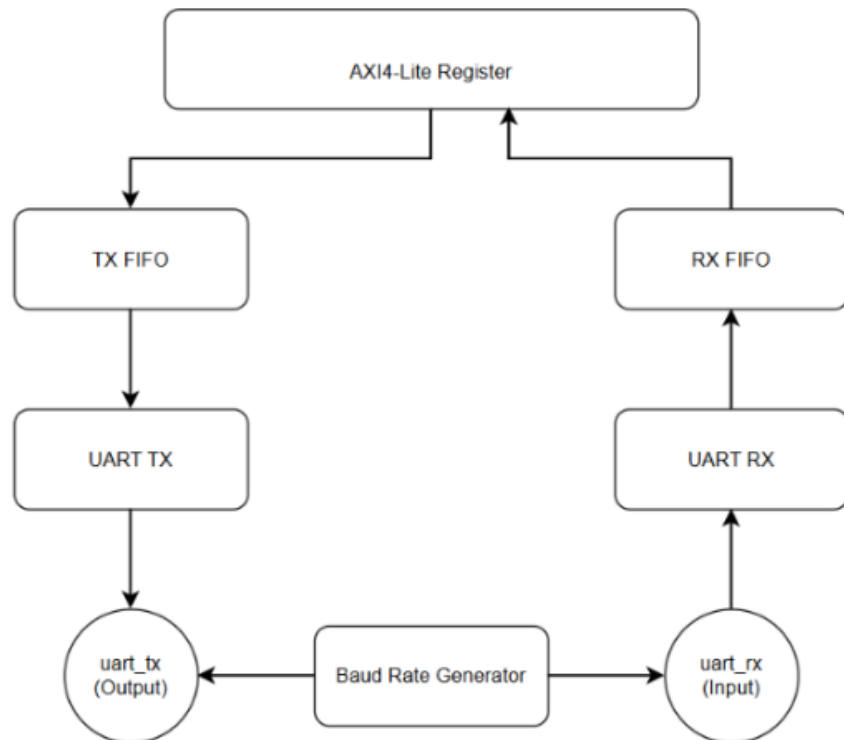
Khối DMA gồm 2 máy trạng thái TX và RX hoạt động hoàn toàn độc lập, có thể chạy song song cùng lúc.

FSM TX tự động đọc từng byte từ RAM và đẩy vào TX FIFO của UART để phát đi

FSM RX tự động lấy từng byte từ RX FIFO của UART và ghi vào bộ nhớ

Cả 2 FSM giao tiếp với bộ nhớ hệ thống qua chuẩn AXI4 master CPU chỉ cần cấu hình địa chỉ và số byte một

Sơ đồ khối - UART



Khối UART là module thực hiện chức năng truyền thông nối tiếp bất đồng bộ giữa hệ thống SoC và thiết bị bên ngoài. Trong đề tài này, UART đóng vai trò là ngoại vi trung gian — nhận lệnh từ CPU qua bus AXI4-Lite, thực hiện việc phát và thu dữ liệu trên đường truyền vật lý, đồng thời phối hợp với khối DMA để truyền dữ liệu khối lớn mà không cần CPU can thiệp từng byte.

Khối UART gồm 6 module con phối hợp với nhau tạo thành một IP hoàn chỉnh

Baud Rate Generator tạo xung lấy mẫu 16 lần tốc độ baud, cung cấp cho cả bộ phát TX và bộ thu RX

UART TX lấy từng byte từ TX FIFO và phát ra chân `uart_tx` theo định dạng 8N1

UART RX nhận tín hiệu từ chân `uart_rx`, giải mã và đẩy byte vào RX FIFO

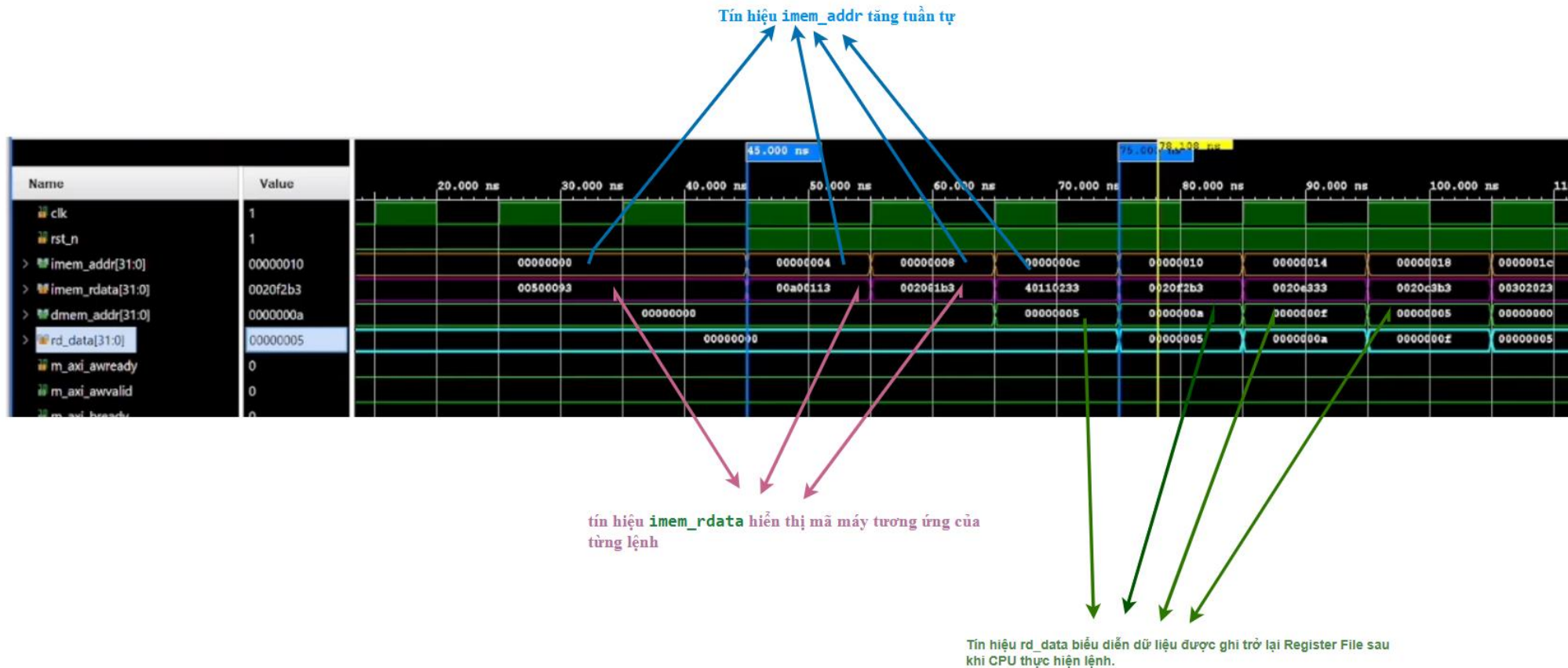
TX FIFO và **RX FIFO** đệm dữ liệu 16 phần tử, giúp CPU và DMA không bị chờ từng byte

AXI4-Lite Register là cầu nối giữa CPU và toàn bộ khối — CPU đọc/ghi 11 thanh ghi để điều khiển UART

Kết quả tổng hợp IP



Kết quả mô phỏng khối SoC RISC-V





Kết quả mô phỏng khối UART

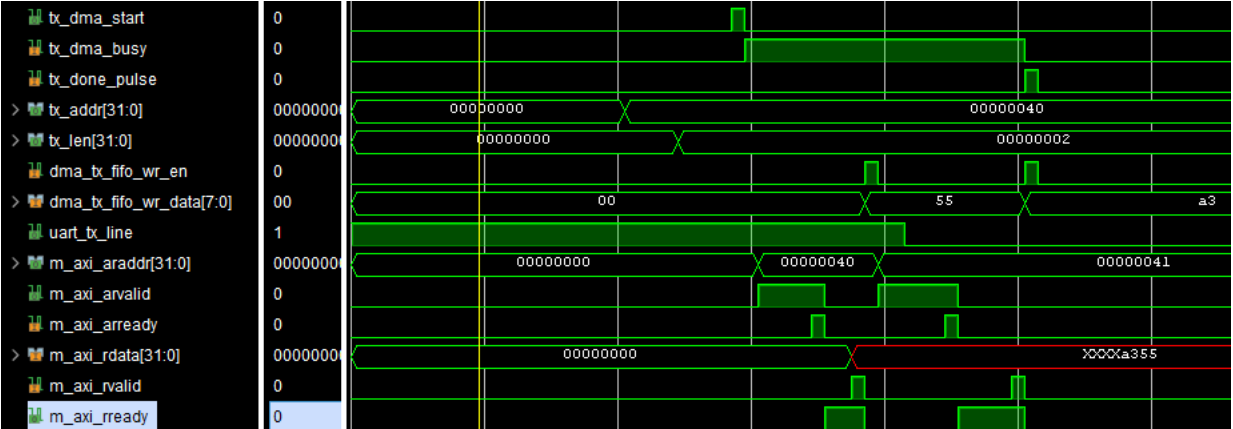


Dạng sóng thể hiện kịch bản trong testbench: CPU ghi 1 byte qua TX_DATA rồi đọc lại qua RX_DATA theo chế độ loopback (chân TX nối thẳng vào chân RX). Đầu tiên CPU thực hiện ghi với `s_axi_awaddr = 0x24` và `s_axi_wdata = 0x00000041` (ký tự 'A'). Tín hiệu `s_axi_awvalid` và `s_axi_wvalid` lên cao rồi xuống sau khi bắt tay AXI hoàn tất, xác nhận byte 0x41 đã được đẩy vào TX FIFO. Ngay sau đó `uart_line` phát từng bit của byte 0x41 ra đường truyền. Vì TX và RX được nối loopback, UART đồng thời thu lại chính byte vừa phát và đẩy vào RX FIFO. Sau khoảng 500 chu kỳ clock, CPU đọc lại với `s_axi_araddr = 0x28` (địa chỉ thanh ghi RX_DATA) và nhận về `s_axi_rdata = 0x00000041`

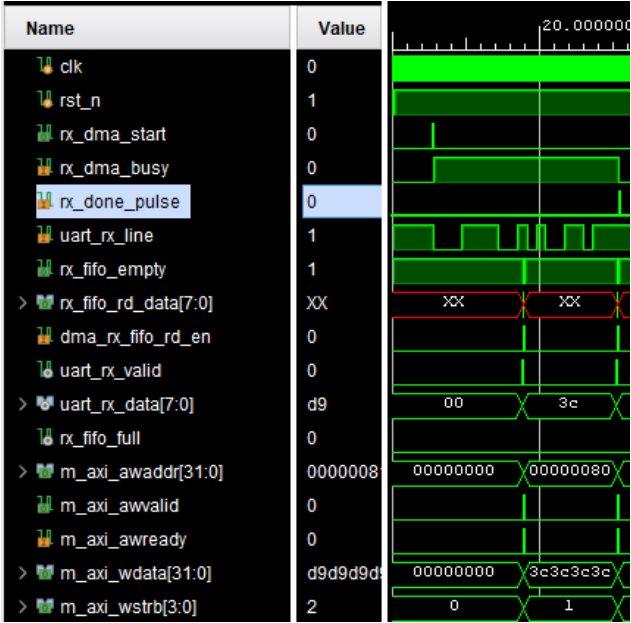


Kết quả mô phỏng khối DMA

Giai đoạn ghi

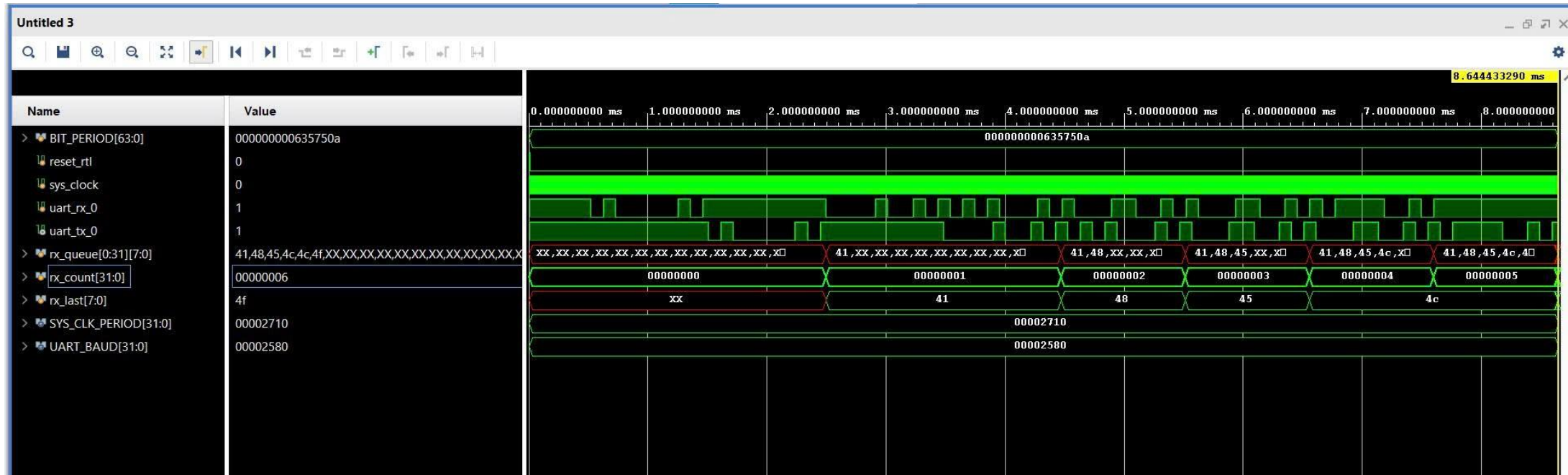


Quá trình ghi dữ liệu



Quá trình đọc dữ liệu

Kết quả mô phỏng toàn hệ thống



Dạng sóng thể hiện kết quả mô phỏng tổng hợp toàn bộ hệ thống. Testbench gửi lần lượt các byte 0x41 ('A'), 0x48 ('H'), 0x45 ('E'), 0x4C ('L'), 0x4C ('L'), 0x4F ('O') vào chân uart_rx_0 và theo dõi phản hồi echo trên uart_tx_0. Tín hiệu rx_count tăng dần từ 0 lên 5 qua từng mốc thời gian, mỗi lần tăng tương ứng với 1 byte được monitor thu về từ uart_tx_0. Tín hiệu rx_last lần lượt hiển thị 0x41 -> 0x48 -> 0x45 -> 0x4C -> 0x4F, đúng thứ tự các byte đã gửi vào.

Thông số tài nguyên sử dụng của hệ thống RISC-V SoC

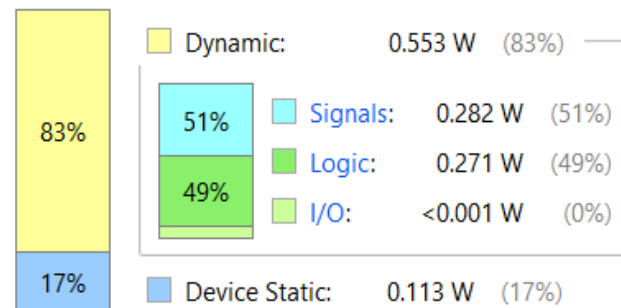
Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	Bonded IOB (200)	BUFGCTRL (32)
▼ N soc_top	4231	2379	1174	513	1517	2183	2048	1	4	1
> I u_cpu (riscv_soc_top)	3725	1619	1174	513	1302	1677	2048	0	0	0
I u_dma (axi4_dma_master)	194	147	0	0	81	194	0	0	0	0
> I u_uart (uart_axi_top)	312	612	0	0	200	312	0	0	0	0

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.666 W
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 32.7°C
 Thermal Margin: 52.3°C (4.4 W)
 Effective θ_{JA} : 11.5°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power





Kết luận – Hướng phát triển

KẾT LUẬN

Đã thiết kế thành công khối UART DMA theo chuẩn giao tiếp AXI tích hợp FIFO cho hệ thống SoC RISC-V.

Hoàn thành các module chính gồm UART TX/RX, FIFO, DMA TX/RX, AXI4-Lite Register và AXI4 Master.

Các module đã được mô phỏng và kiểm chứng chức năng, đáp ứng đúng yêu cầu thiết kế.

Kết quả tổng hợp cho thấy hệ thống sử dụng ít tài nguyên logic FPGA, thuận lợi cho việc tích hợp và mở rộng.

HƯỚNG PHÁT TRIỂN

Hoàn thiện việc tích hợp toàn bộ hệ thống trên SoC RISC-V và kiểm thử trên phần cứng FPGA.

Tối ưu hiệu năng của DMA nhằm tăng tốc độ truyền dữ liệu.

Tài liệu tham khảo

- [1] A. Waterman and K. Asanović, Eds., The RISC-V Instruction Set Manual, Volume I: User-Level ISA, RISC-V Foundation, 2019.
- [2] D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware/Software Interface, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2020.
- [3] ARM Limited, "AMBA AXI and ACE Protocol Specification," ARM IHI 0022, 2021.
- [4] Xilinx Inc., "Zynq-7000 SoC Technical Reference Manual," UG585, 2021.
- [5] Xilinx Inc., "Vivado Design Suite User Guide: Synthesis," UG901, 2022.
- [6] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2017.
- [7] Telecommunications Industry Association, "Interface Between Data Terminal Equipment and Data Circuit-Terminating Equipment Employing Serial Binary Data Interchange," TIA/EIA-232-F, 1997.
- [8] ARM Ltd., "AMBA AXI4-Stream Protocol Specification," IHI0051A, ARM Holdings, 2010.
- [9] S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.

Group photo

